
PL-200 Parameters & the Fabrikam Solution

Contents

Fabrikam	3
The Business Story	3
Reverse Engineer the Design, Diagram	4
What The Model-Driven App Is Doing	5
What The Flows Are Likely Doing.....	6
What The Environment Variables Tell You	7
Design Review	8
What is the user trying to accomplish?	8
What is the system of record?	8
Where does the user interact?	8
What automates work?	8
What controls access?	8
What moves between environments?	9
How To "Peek"	9
One-Sentence Reverse Engineering Summary	11
Variables & Parameter Thinking	12
Variables and Parameters = The Baton.....	13
Simple Distinction	14
Parameter	14
Variable	14
When Are Variables NOT Needed?.....	15
Where Variables Become Useful	16
Power Automate	16
Canvas Apps	17
Power Pages	17
Do They Have A Central Viewing Spot?	18
Power Automate	18
Environment Variables.....	19

Do They Always Show In The Lightning Bolt?	20
Do They Have Scope?	21
Environment Scope	21
App Scope	21
Flow Scope	21
Page Scope	22
Best Design Thoughts.....	22
1. Environment Moves	22
2. Passing Data	23
3. Passing Files	23
4. Configuration	24
The Five Levels Of "Stuff"	24
Remember One Thing	25
Simplest Possible Parameter Flow	26
Simple Add Row Flow	28
Power Automate flow w Params	30
🔗 The Six Input Types	31
📄 Demo: A Purchase Request Flow.....	32
PL-200 Git Labs & Flows	42
Where This Appears In The Labs.....	43
Is There Something In Fabrikam?	44
If You Want To "Peek In Code"	45
The Progression of What to Learn/When	47

Fabrikam

On the Fabrikam - how would you describe what the apps or flows do? Design discussions on that are not part of the course it seems but from a Peek at it point of view would have value?

Copilot

Honestly, I think this is one of the biggest missed opportunities in the PL-200 training.

The labs teach:

How to build

but spend almost no time on:

What was built?

Why was it built?

How do the pieces fit together?

The Fabrikam solution actually has a fairly decent business story hidden inside it.

The Business Story

Fabrikam is presented as:

Environmental Consulting Company

They manage:

Programs

Projects

Outcomes

Stakeholders

for environmental initiatives.

Think:

Water Quality Program

Project A

Project B

Project C

instead of:

CRM Sales Leads

or

Help Desk Tickets

The labs import a solution containing tables, apps, business rules, roles, flows, dashboards, and other components to support that scenario.github+1

Reverse Engineer the Design, Diagram

Program

|

+-- Projects

|

+-- Outcomes

|

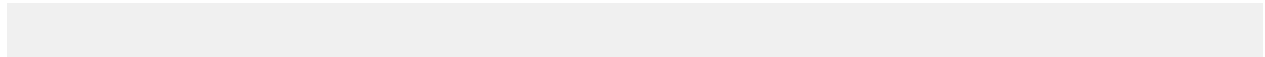
+-- Measurements

- A consultant enters data.
- Managers review progress.
- Dashboards summarize results.
- Flows automate notifications and status updates.

What The Model-Driven App Is Doing

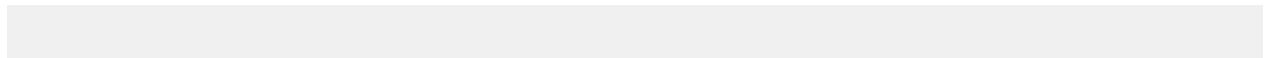
The Model-Driven App is probably the primary working application.

Think:



Environmental Management System

Users can:



Create Program

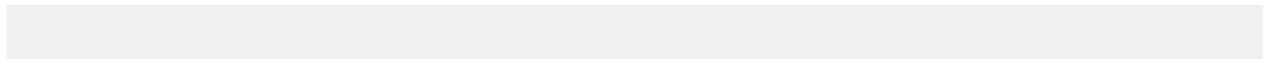
Create Project

Update Outcome

Review Progress

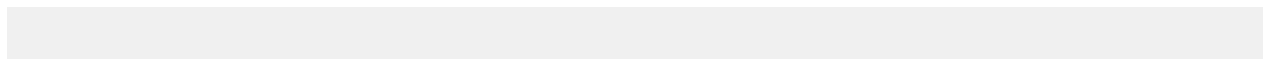
Run Dashboards

The forms and views labs are really teaching:



How users interact with these records

not just:



How to move fields around on a screen

What The Flows Are Likely Doing

Although the labs don't stop and explain this well, the flows generally exist to automate the process.

I would expect patterns like:

Outcome Created

↓

Notification Sent

or

Project Updated

↓

Status Changed

or

Milestone Reached

↓

Create Follow-up Activity

The purpose of the flow is usually:

Reduce manual work

rather than:

Store data

Dataverse stores the data.

Flows move the process forward.

What The Environment Variables Tell You

The screenshot you posted earlier is actually a great clue.

Variables such as:

[Redacted]

Outcome Form Link

LOB Guid List

tell me the developers wanted configuration separated from code.

In traditional development:

[Redacted]

appsettings.json

config.xml

web.config

In Power Platform:

[Redacted]

Environment Variables

Inside the solution.

That suggests the solution was designed for movement between:

[Redacted]

Dev

Test

Prod

..

without modifying the flows themselves.

Design Review

What is the user trying to accomplish?

Track environmental programs

What is the system of record?

Dataverse

Where does the user interact?

Model Driven App

What automates work?

Power Automate

What controls access?

Security Roles

..

What moves between environments?

Solutions

Environment Variables

Connection References

How To "Peek"

When opening Fabrikam, I'd explore in this order:

1. Tables

Ask:

What business entities exist?

2. Relationships

Ask:

How are entities connected?

3. Model Driven App

Ask:

What workflows do users follow?

4. Flows

Ask:

What business events trigger automation?

5. Environment Variables

Ask:

What configuration was externalized?

6. Dashboards

Ask:

What information does management care about?

One-Sentence Reverse Engineering Summary

The Fabrikam solution is not really a collection of forms, flows, and tables—it is a

1. Small environmental project-management application
2. Where Dataverse stores program and outcome data,
3. The Model-Driven App provides the user interface,
4. Power Automate handles process automation,
5. Security roles control access, and
6. Dashboards summarize progress.

The labs teach the components individually, but the real value comes from understanding how all those components cooperate as a single business application.

Variables & Parameter Thinking

Create **variables or parameters** in Power Pages when managing dynamic, environment-specific configurations (like API keys or URLs), passing query strings between pages using [Liquid objects](#), or exchanging data with a [Power Automate cloud flow](#). [1, 2, 3, 4, 5]

When to Create Variables/Parameters

- **Site Settings (Environment Variables):** Use these when you want to avoid hardcoding values and need site settings (like connection strings or third-party integrations) to adapt across dev, test, and production. [1]
- **Query Parameters:** Use these if you need to pass data (like specific record IDs) between your web pages via the browser's URL. [1]
- **Flow Parameters:** Use these when you trigger a cloud flow from a form submission and need to pass user-defined inputs or Dataverse GUIDs back and forth. [1, 2, 3]

How to View What is Already Created

- **In Power Pages (Environment Variables):** Open the [Power Pages Management app](#), navigate to **Environment Variables**, and check the definitions table. To view and modify how they apply to site settings, go to the **Site Settings** view and check the Source. [1]
- **In Power Automate Cloud Flows:** All inputs, outputs, and initialized variables are visible as dynamic content inside the flow designer. You can track their values by reviewing previous flow **Run History** in the Power Automate Maker Portal. [1, 2, 3, 4]
- **In Power Apps Canvas Apps:** If your Power Pages site connects to an embedded canvas app, you can view all active variables by going to the **Content** tab in the Studio and clicking on **Variables**.

Variables and Parameters = The Baton

"LETS" (Least Expensive Training Speak) explanation.

A great beginner analogy is:

```
1 Data = Baton
2
3 Variables = Hands holding the baton
4
5 Parameters = Hand-off points
6
7 Flow/App/Page = Marathon runners
8
9 Business Process = Race course
```

The application's job is to move data from:

```
1 User
2 ↓
3 Page
4 ↓
5 App
6 ↓
7 Flow
8 ↓
9 Dataverse
10 ↓
11 Report
```

- Without dropping the baton.

Simple Distinction

Parameter

A value coming into something.

Example:

```
1 Flow Input
2
3 CustomerID
```

The caller supplies it.

Think:

```
1 Function Argument
```

Variable

A value stored during processing.

Example:

```
1 CurrentCustomer
2 CurrentStatus
3 TotalCost
```

Think:

```
1 Scratch Pad
```

for the running process.

When Are Variables NOT Needed?

Many Power Platform solutions use almost none.

Example:

```
1 User enters form
2   ↓
3 Dataverse saves row
4   ↓
5 Done
```

No variable required.

Power Pages:

```
1 Display Dataverse table
```

No variable required.

Model-Driven App:

1 Open Form

2 Edit Record

3 Save Record

No variable required.

The platform already manages the data.

Where Variables Become Useful

Power Automate

Most common location.

Most common location.

- 1 Input Parameter
- 2 ↓
- 3 Variable
- 4 ↓
- 5 Loop
- 6 ↓
- 7 Approval
- 8 ↓
- 9 Update Dataverse

Useful for:

- 1 Counters
- 2 Flags
- 3 Running Totals
- 4 Arrays
- 5 Strings

Canvas Apps

Extremely common.

Examples:

```
1 Set(CurrentProject,...)
2
3 Set(CurrentUser,...)
4
5 UpdateContext(...)
```

Used for:

```
1 Screen State
2 User Choices
3 Navigation
4 Temporary Values
```

Power Pages

Less common.

```
1 Query String
2
3 Record ID
4
5 Site Settings
6
7 Liquid Variables
```

rather than traditional variables.

Do They Have A Central Viewing Spot?

Power Automate

Sort of.

Inside a flow:

1 Variables

appear as actions:

- 1 Initialize Variable
- 2
- 3 Set Variable
- 4
- 5 Append To Variable

and later as Dynamic Content.

- There is no single tenant-wide variable list.

Environment Variables

These do have a centralized location.

These do have a centralized location.

- 1 Solution
- 2 ↓
- 3 Environment Variables

These are the closest thing to:

- 1 Global Configuration

Examples:

- 1 API URL
- 2
- 3 SharePoint URL
- 4
- 5 Tenant ID
- 6
- 7 Default Email Address

Do They Always Show In The Lightning Bolt?

Not always.

Think of Dynamic Content as:

1 Available Batons

- at the current point in the process.
- If the flow knows about a value, it usually appears.

Examples:

1 Trigger Inputs

2

3 Previous Action Outputs

4

5 Variables

6

7 Environment Variables

8``

appear as Dynamic Content.

But:

1 Future Values

cannot appear.

The flow cannot reference:

1 Step 8

while you're still in:

1 Step 3

Do They Have Scope?

Absolutely.

This is a very important concept.

Environment Scope

1 Environment Variable

Available to solution components.

Think:

1 Global Configuration

App Scope

1 Canvas Variable

Only exists inside the app.

Flow Scope

1 Flow Variable

Only exists while the flow runs.

When flow ends:

1 Variable disappears

Page Scope

Power Pages:

1 URL Parameters

2 Liquid Objects

exist only for that page request.

Best Design Thoughts

1. Environment Moves

Excellent use case.

Instead of:

```
1 https://dev.contoso.com
```

hard coded everywhere:

Use:

```
1 Environment Variable
```

Then:

```
1 Dev
2 Test
3 Prod
```

can each use different values.

2. Passing Data

Very common.

Examples:

```
1 Canvas App
2   ↓
3 ProjectID
4   ↓
5 Flow
6   ↓
7 Dataverse
```

or

```
1 Power Page
2   ↓
3 RecordID
4   ↓
5 Flow
```

3. Passing Files

Usually not via variables.

Normally:

```
1 File
2   ↓
3 SharePoint
4
5 OneDrive
6
7 Dataverse File Column
8
9 Blob Storage
```

Then pass:

```
1 URL
2 ID
3 Path
```

instead.

4. Configuration

One of the best uses.

Example:

```
1 SupportEmail
2
3 ManagerGroup
4
5 ApprovalPath
```

- stored once.
- Referenced everywhere.

The Five Levels Of "Stuff"

```
1 Literal Value
2
3     "Hello"
4
5     ↓
6
7 Parameter
8
9     CustomerID
10
11    ↓
12
13 Variable
14
15     CurrentCustomer
16
17    ↓
18
19 Environment Variable
20
21     ApiEndpoint
22
23    ↓
24
25 Dataverse
26
27     Permanent Storage
```


Remember One Thing

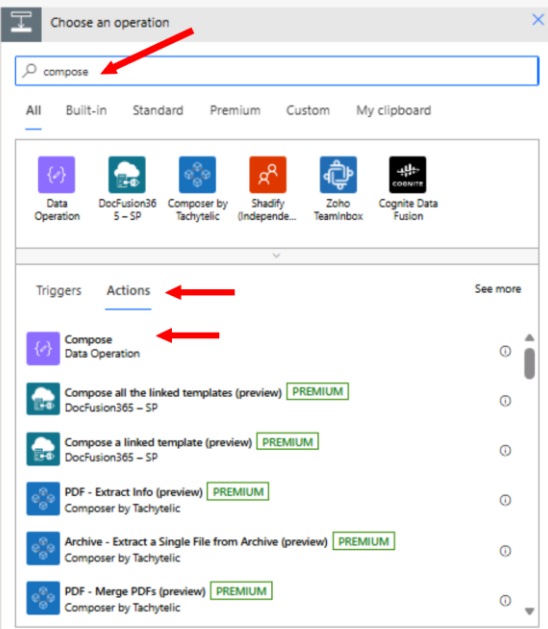
- 1 Parameters
- 2 bring data in
- 3
- 4 Variables
- 5 hold data temporarily
- 6
- 7 Environment Variables
- 8 store configuration
- 9
- 10 Dataverse
- 11 stores business data permanently

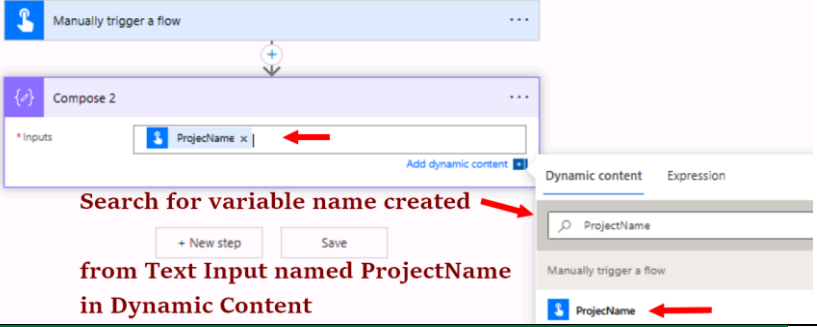
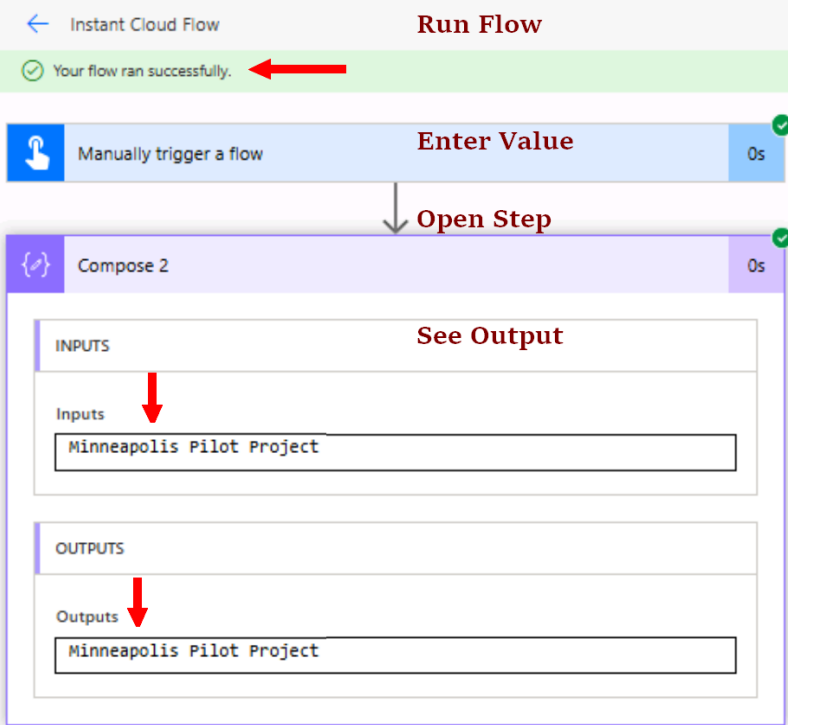
That's about 90% of what a PL-200 consultant needs to understand before diving into apps, flows, Power Pages, and solutions.

Simplest Possible Parameter Flow

[Use data operations in Power Automate - Power Automate | Microsoft Learn](#)

Create something so small that the concept becomes obvious.

Flow 1. Create: a. Instant Cloud Flow 2. Trigger: a. Manually trigger a flow 3. Add Input: a. Text 4. Name: a. ProjectName Now Flow has: ProjectName as a parameter.	
Action 1. Add: a. Compose	

2. Input: a. ProjectName 3. (dynamic content) b. Save.	 Search for variable name created from Text Input named ProjectName in Dynamic Content
Test 1. Run. 2. Enter: a. Minneapolis Pilot Project Result: Compose Output Minneapolis Pilot Project	 Run Flow Your flow ran successfully. Enter Value Open Step See Output INPUTS Inputs Minneapolis Pilot Project OUTPUTS Outputs Minneapolis Pilot Project

Now the student sees:

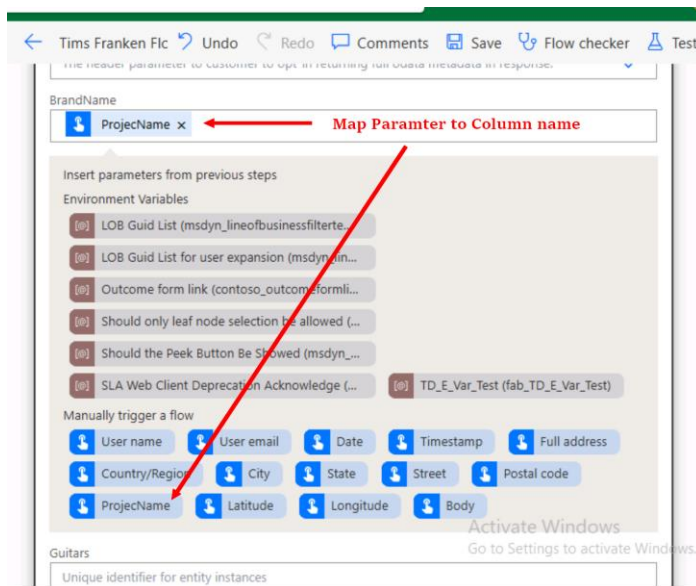
- Input Parameter
- ↓
- Flow
- ↓
- Output

..

working.

Simple Add Row Flow

1. Instead of Compose:
 - a. Add new row
2. Dataverse table:
 - a. Training Parameters
3. Column:
 - a. Project Name
4. Map:
 - a. Project Name
5. =
 - a. ProjectName parameter



6. Run flow.
7. Enter:
 - a. PL200 Test
8. Dataverse receives:
 - a. PL200 Test
9. **Now you've demonstrated:**
 - Parameter
 - ↓
 - Flow
 - ↓
 - Dataverse

which is far more realistic.

Flow Types

- **Dataflows** are designed for **data preparation** (extracting, transforming, and loading data).
- **Cloud flows** (in Power Automate) are designed for **process automation** (triggering actions and workflows based on events or schedules).

In short, dataflows prepare your data for analysis, while cloud flows automate tasks and notifications. [1, 2, 3, 4]

Core Differences

- **Primary Function:** Dataflows perform heavy data cleaning and aggregation to load into repositories like Dataverse or Azure Data Lake. Cloud flows connect disparate apps (like sending a Microsoft Teams message or saving a file to SharePoint when a record is updated). [1, 3, 4]
- **Engine:** Dataflows use the [Power Query](#) engine, utilizing a graphical editor to shape large datasets. Cloud flows use logical triggers and conditions (if this, then that). [1, 4]
- **User Persona:** Dataflows are tailored for data analysts building models for reporting. Cloud flows are built for citizen developers or administrators looking to streamline daily business processes. [1, 2]

How They Work Together

Instead of competing, these two tools are frequently paired together. For example, you can set up a **cloud flow** to automatically trigger a **dataflow's** refresh on a daily schedule and, once that dataflow finishes loading the processed data, send a notification or report to your team. [1]

To optimize these tasks and prevent coding mistakes when building out these systems, it is recommended to follow established design and development guidelines [Avoid anti-patterns - Power Automate - Microsoft Learn](#). [1]

Power Automate flow w Params

Parameters allow you to pass dynamic data into your flow each time it runs, making your automation flexible and reusable. [1, 2, 3]

To create a Power Automate flow with parameters, you must

- **define inputs directly inside a manually triggered cloud flow**
- Or **initialize variables that request inputs from a calling app like [Power Apps](#).**

Here is how to set up parameters using the two most common methods.

Method 1: Using a Manual Trigger (Instant Cloud Flow)

This is the easiest way to prompt a user for parameters whenever they start the flow manually. [1]

1. **Sign in:** Go to the [Power Automate Portal](#).
2. **Create Flow:** Click **Create > Instant cloud flow**.
3. **Choose Trigger:** Name your flow, select **Manually trigger a flow**, and click **Create**.
4. **Add Parameter:** Inside the trigger card, click **Add an input**.
5. **Select Type:** Choose your data type (e.g., *Text, Number, Boolean, File*).
6. **Name Input:** Change the default "Input" name to a descriptive parameter title (e.g., *EmployeeEmail*).
7. **Use Parameter:** Click **New step**, choose an action (like *Send an email*), click into a field, and select your parameter from the **Dynamic content** window. [1, 2, 3, 4, 5]

Method 2: Passing Parameters From Power Apps

Use this method if you want a custom canvas application to send data directly into your automation workflow. [1, 2]

1. **Create Flow:** In the Power Automate Portal, create a new **Instant cloud flow**. [1, 2]
2. **Choose Trigger:** Select the **PowerApps** trigger and click **Create**. [1]
3. **Add Action:** Click **New step** and search for **Initialize variable**. [1, 2]
4. **Configure Variable:**
 - Give your variable a unique name.
 - Set the **Type** (e.g., *String* or *Integer*). [1, 2, 3]
5. **Link Parameter:** Click inside the **Value** box, look at the Dynamic Content panel, and click **Ask in Power Apps**.

- *Note: Power Automate will automatically generate a parameter matching your variable name.* [\[1, 2, 3, 4, 5\]](#)
6. **Call from App:** Save the flow, open your app in [Power Apps Studio](#), connect the flow, and pass the text/data inside the formula bar (e.g., YourFlowName.Run(TextInput1.Text)). [\[1, 2, 3\]](#)

Method 3: Environmental Parameters (Using Solutions)

If you want global environment variables that change based on whether the flow is in Development, Testing, or Production, use Solution parameters. [\[1\]](#)

1. **Open Solutions:** Click **Solutions** in the left sidebar and open your current working solution.
2. **New Variable:** Click **New > More > Environment variable**.
3. **Configure:** Define the schema name, data type, and default value.
4. **Insert into Flow:** Open a cloud flow *inside that same solution*, and your environment variable will show up directly in the **Dynamic content** pane under parameters. [\[1, 2, 3, 4, 5\]](#)

If you want to look at more advanced implementations, review the official documentation on [defining input and output parameters](#) or [creating child workflows](#). [\[1, 2\]](#)

Would you like to know how to pass these parameters to a **child flow**, or are you looking to trigger this flow from a specific application like **Power Pages** or **SharePoint**? [\[1, 2, 3\]](#)

What Are Power Automate Input Parameters?

Here's the thing: most flows I build in the real world are automated. They kick off on their own when an item gets created or modified in a SharePoint list or library, and end users never interact with them directly.

But sometimes a *person* needs to start a process — an approval, a file review, a request. That's a **manually triggered flow**, and that's where Power Automate input parameters come in. Instead of just clicking "Run flow," the person gets a little form to fill out first, and everything they type becomes data you can use anywhere in the flow.

A few important things to know up front:

- Input parameters only apply to flows that a person starts — the **Manually trigger a flow** trigger and the SharePoint **For a selected item / For a selected file** trigger.
- There's no visual form designer. You just add inputs, and each one automatically becomes a field in the Run flow panel. The order you add them is the order they appear (and no, you can't rearrange them — you'd have to delete and re-add them).
- Every input is a **required field** in the form by default, but you can make any of them optional.

The Six Input Types

When you add an input to your trigger, you get six types to choose from:

1. **Text** — a simple text box
2. **Yes/No** — a boolean toggle
3. **File** — lets the person attach a file
4. **Email** — this is really a *people picker*, which is handy
5. **Number** — numbers only
6. **Date** — a date picker

Now here's the hidden menu I mentioned in the video title. Click the ... (**ellipsis**) next to any text input, and you'll find some powerful options:

- **Make the field optional** (or required again)
- **Add a drop-down list of options** — the person picks one
- **Add a multi-select list of options** — the person can pick several

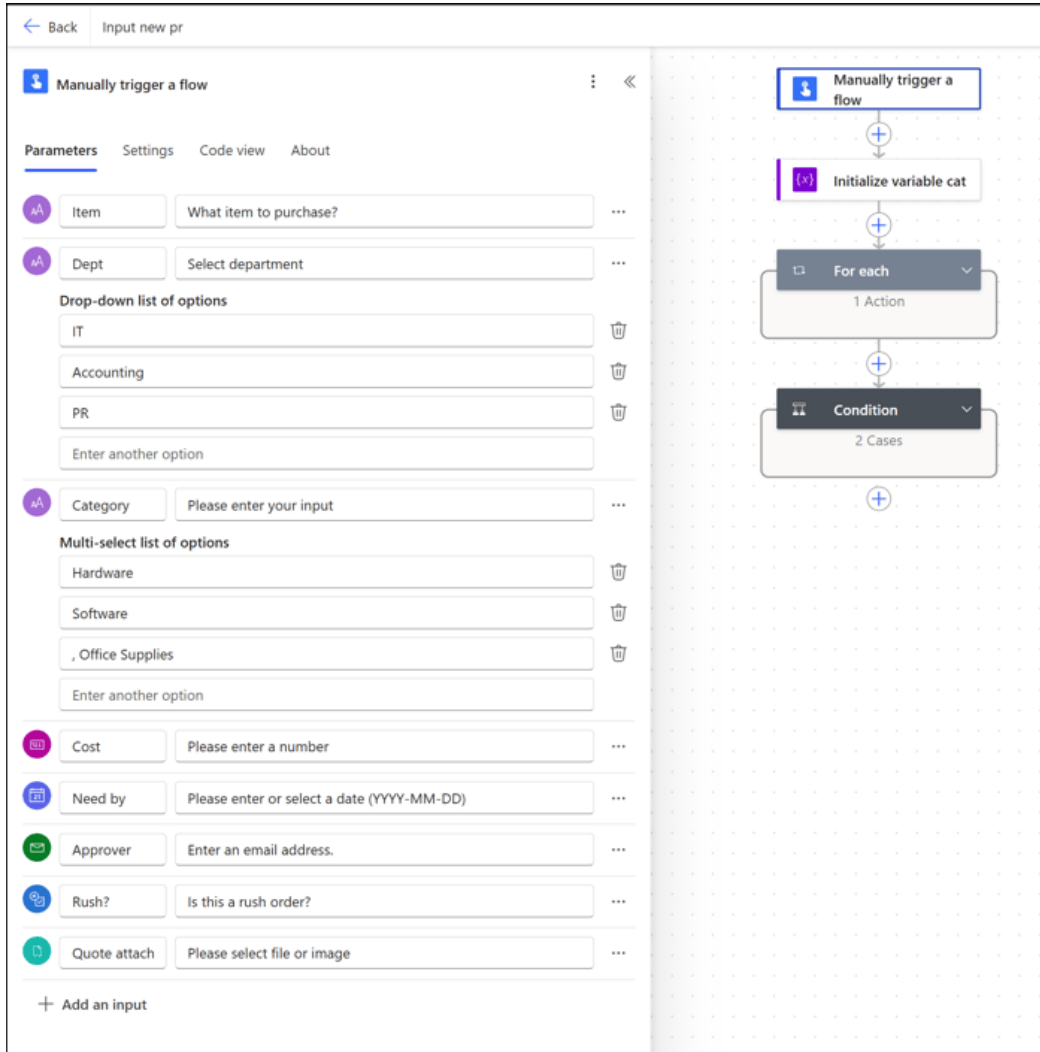
That little menu is easy to miss, and it's what turns a plain text box into real data validation. No more hoping people type the department name correctly — give them a drop-down.

Demo: A Purchase Request Flow

To show off all the input types, I built a purchase request form using nothing but Power Automate input parameters:

1. **What is the item?** — Text
2. **Department** — Text with a drop-down list (IT, Accounting, PR)
3. **Category** — Text with a multi-select list (hardware, software, and so on)
4. **Cost** — Number
5. **Need by** — Date
6. **Approver** — Email (remember, it's a people picker)
7. **Is this a rush order?** — Yes/No
8. **Quote attachment** — File, set to *optional*

Flow screenshots



The screenshot displays a flow builder interface. On the left, the 'Parameters' tab is active, showing a list of input fields for a 'Manually trigger a flow' action. The parameters include:

- Item:** Text input with placeholder 'What item to purchase?'
- Dept:** Text input with placeholder 'Select department'
- Drop-down list of options:** A list with 'IT', 'Accounting', and 'PR', plus an 'Enter another option' field.
- Category:** Text input with placeholder 'Please enter your input'
- Multi-select list of options:** A list with 'Hardware', 'Software', and 'Office Supplies', plus an 'Enter another option' field.
- Cost:** Text input with placeholder 'Please enter a number'
- Need by:** Text input with placeholder 'Please enter or select a date (YYYY-MM-DD)'
- Approver:** Text input with placeholder 'Enter an email address.'
- Rush?:** Text input with placeholder 'Is this a rush order?'
- Quote attach:** Text input with placeholder 'Please select file or image'

At the bottom of the parameters list is a '+ Add an input' button. On the right, the flow diagram shows the sequence of steps:

- Manually trigger a flow
- Initialize variable cat
- For each (1 Action)
- Condition (2 Cases)

Then I added a **Condition** to check whether "Is this a rush order?" is equal to true.

If it is, the flow sends an email to the approver with the importance set to **High** and "RUSH" in the subject line.

If not, it sends a normal one.

That's the part I love about input parameters — they don't just fill in blanks in an email.

They can drive the logic of your flow and even decide who the email goes to.

Here's what the initiation form looks like when you click to manually start the flow:

Condition

Parameters Settings Code view About

Condition expression *

Provide the values to compare and select the operator to use.

And

Rush? X

is equal to

1 true X

Choose a value

is equal to

Choose a value

+ Add row

Condition

True

False

Send an email (V2)

1 Action

Run flow

Input new pr

Owner: Laura Rogers

Item *

What item to purchase?

Dept *

Select department

Category *

Please enter your input

Cost *

Please enter a number

Need by

Please enter or select a date (YYYY-MM-DD)

Approver *

Enter an email address.

Rush?

Quote attachment

Please select file or image

Import

This flow uses Office 365 Outlook.

[Review connections and actions](#)

Run flow

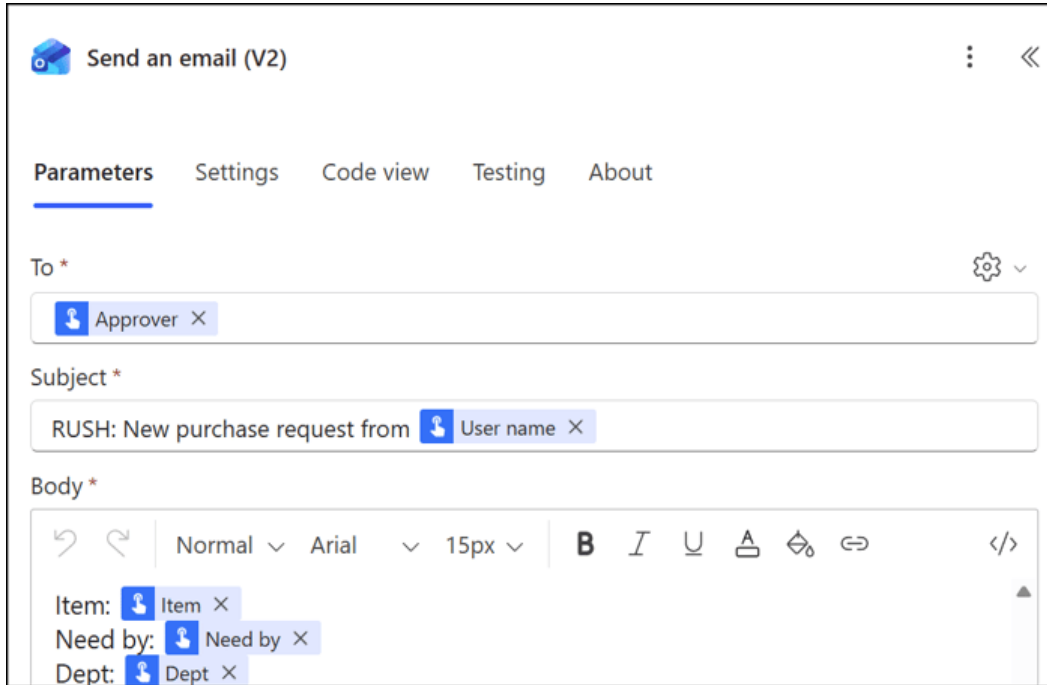
Cancel

Bonus Data You Get for Free

- When someone fills out the form, the trigger gives you more than just their answers.

- You automatically get dynamic content for the person's **name**, **email address**, a **timestamp**, and even their **location** — city, state, postal code, latitude and longitude from their browser or mobile device.

I used the person's name in my email subject line: "New purchase request from so-and-so." [enter their display name]



Send an email (V2)

Parameters Settings Code view Testing About

To *

Approver X

Subject *

RUSH: New purchase request from User name X

Body *

Normal Arial 15px B I U A ↺ ↻

Item: Item X

Need by: Need by X

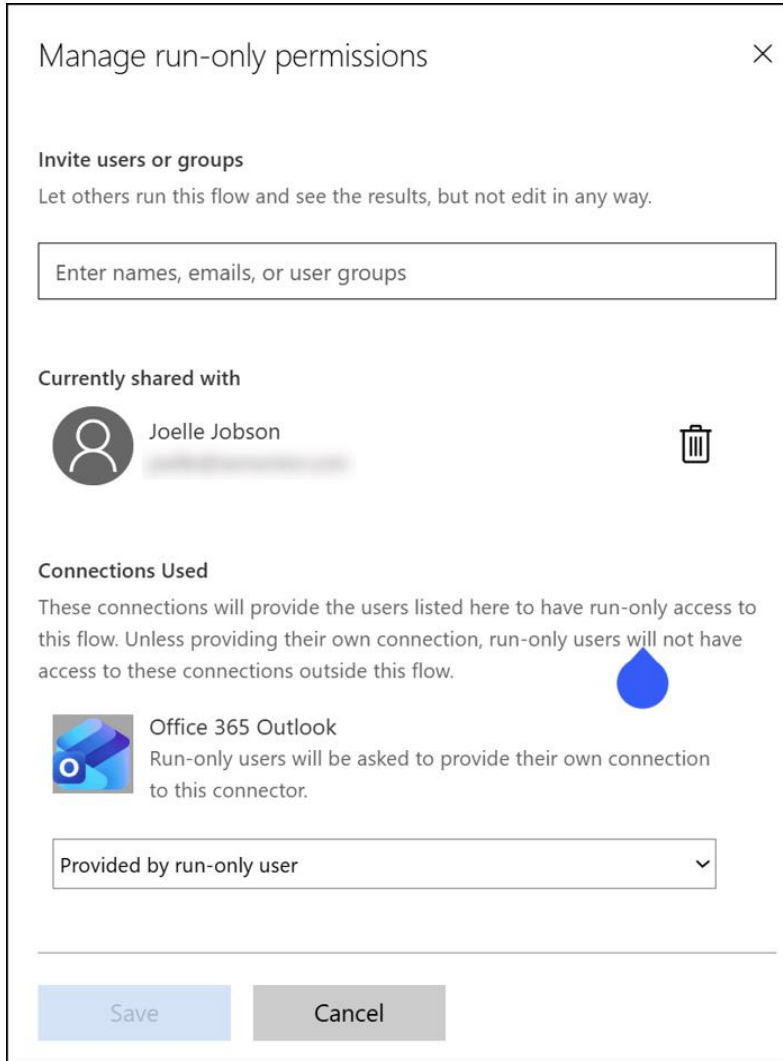
Dept: Dept X

Who Gets the Button? Run-Only Users

By default, you're the owner of your flow and you're the *only* one who can run it. To hand the button to other people, open the flow's details page and add **run-only users** — individuals or even a whole group.

A few things I want you to know about this:

- When you add someone, they get an **email with a link** to manually trigger the flow. You can grab that hyperlink and put it anywhere — like a button right on your SharePoint home page — so people don't have to go digging through Power Automate to find it.
- Run-only users also see the flow in the **Power Automate mobile app**, under Instant Flows. They tap it, fill out the same little form, and go. Great for folks out in the field.
- Here's the powerful part: when you add run-only users, you can choose whose **connections** the flow uses. If the flow saves data to a SharePoint list the person doesn't have access to, you can set the SharePoint connection to run as *you*, the flow creator. The person filling out the form never needs access to that list or library. This capability is only available on manually triggered flows — automated flows don't have run-only users at all. This example only shows an Outlook connector, but it will list all of the connectors used in your flow.



Manage run-only permissions

Invite users or groups
Let others run this flow and see the results, but not edit in any way.

Enter names, emails, or user groups

Currently shared with

 Joelle Jobson 

Connections Used
These connections will provide the users listed here to have run-only access to this flow. Unless providing their own connection, run-only users will not have access to these connections outside this flow.

 Office 365 Outlook
Run-only users will be asked to provide their own connection to this connector.

Provided by run-only user

Save Cancel

If you've been around SharePoint as long as I have, that last one might ring a bell — it's the modern answer to the old "impersonation step" we had in SharePoint Designer workflows.

Also note that in this demo, I didn't actually SAVE or add the data anywhere. I only sent it in an email. So if you wanted to save the information that was filled in, you'd need to add an action to create an item in SharePoint or your database of choice like Dataverse.

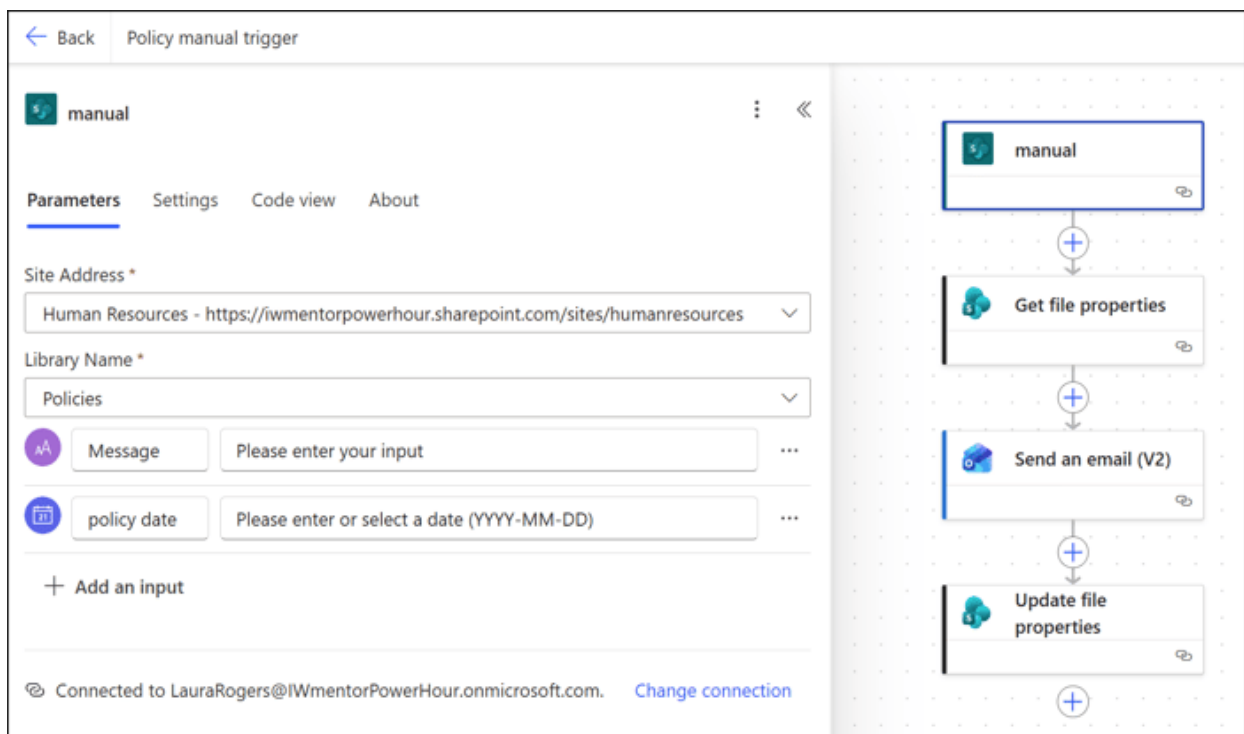
Power Automate Input Parameters in SharePoint

Now let's bring this into SharePoint, because Power Automate input parameters aren't just for the manual trigger. The **For a selected item** trigger (for lists) and **For a selected file** trigger (for libraries) let people run your flow on a specific item, right from SharePoint — and yes, you can add the same inputs to these triggers.

I built one on my Policies library:

1. Create a new flow with the **For a selected file** trigger, and pick your site and library.
2. Click **Add an input** and add what you need. I added a **Message** (text) and a **Policy date** (date).

- Now, here's the gotcha: this trigger does *not* give you the metadata from the item. You only get the basics plus your inputs. So always add a **Get file properties** action (or **Get item** for a list) right after the trigger, using the ID from the trigger.
I honestly consider this a bug. Why wouldn't the trigger just include the item's fields? But once you add it, all your columns show up as dynamic content.
- From there, I sent an email to the **Policy owner** (a column in my library) with the message from the input and a link to the file.
Tip: the "Link to item" dynamic content is a big ugly URL, so I wrapped it in a simple HTML hyperlink in the email body that just says "Click here to open the file."
- I also added an **Update file properties** action to take the Policy date the person filled in and stamp it right onto the file's metadata.



To run it, you select a file in the library, open the **Integrate** menu, and click your flow.

Policies ▾

Open ▾ Share Copy link Delete Pin to top Add to favorites Add shortcut ▾ Download

Name ▾		Open ▾	Modified By ▾	Policy owner ▾	Policy Date ▾
Acceptable Use.docx		Open	Laura Rogers	Laura Rogers	6/17/2026
Attendance.docx		Preview	Laura Rogers	Joelle Jobson	
Bring Your Own Device.docx		Share	Laura Rogers	Brenda Cox	
Code of Conduct.docx		Copy link	Laura Rogers	Laura Rogers	6/15/2026
Contoso Dress Code.docx	...	Manage access	Laura Rogers	Joelle Jobson	
Cybersecurity.docx		Delete	Laura Rogers	Brenda Cox	
Equal Opportunity.docx		Add to favorites	Laura Rogers	Laura Rogers	
Flexible Work.docx		Add shortcut	Laura Rogers	Joelle Jobson	
Harassment.docx		Download	Laura Rogers	Brenda Cox	
Health and Safety.docx		Rename	Laura Rogers	Laura Rogers	
Leave of Absence.docx		Pin to top	Laura Rogers	Joelle Jobson	
New Policy for Vacation.docx		Move to	Laura Rogers	Brenda Cox	
Paid Time Off.docx		Copy to	Laura Rogers	Laura Rogers	

Integrate ▾

Flows ▾

- Request sign-off
- Set a reminder ▾
- Policy manual trigger
- Cox
- Laura Rogers

The input panel slides out on the right side of the screen, the person fills it in, and off it goes. In my demo, the email arrived and the Policy date column updated almost instantly.

The screenshot shows a SharePoint 'Policies' library. The table lists various policy documents with columns for Name, Modified, Modified By, and Policy owner. The 'Bring Your Own Device.docx' file is selected. A 'Run flow' dialog box is open on the right, showing a 'Policy manual trigger' flow. The dialog includes a 'Message' input field with the placeholder 'Please enter your input' and a 'policy date' input field with the placeholder 'Please enter or select a date (YYYY-MM-DD)'. At the bottom of the dialog, it states 'This flow uses SharePoint, and Office 365 Outlook.' and provides a link to 'Review connections and actions'. There are 'Run flow' and 'Cancel' buttons at the bottom right of the dialog.

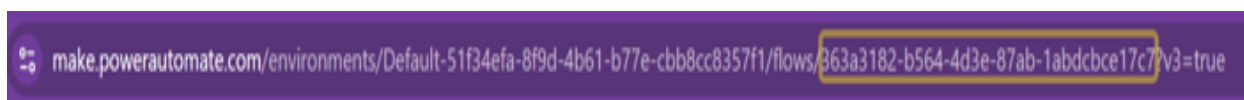
Name	Modified	Modified By	Policy owner
Acceptable Use.docx	Yesterday at 11...	Laura Rogers	Laura Rogers
Attendance.docx	May 13	Laura Rogers	Joelle Johnson
Bring Your Own Device.docx	May 13	Laura Rogers	Brenda Cox
Code of Conduct.docx	Yesterday at 11...	Laura Rogers	Laura Rogers
Contoso Dress Code.docx	May 13	Laura Rogers	Joelle Johnson
Cybersecurity.docx	May 13	Laura Rogers	Brenda Cox
Equal Opportunity.docx	May 13	Laura Rogers	Laura Rogers
Flexible Work.docx	May 13	Laura Rogers	Joelle Johnson
Harassment.docx	May 13	Laura Rogers	Brenda Cox
Health and Safety.docx	May 13	Laura Rogers	Laura Rogers
Leave of Absence.docx	May 13	Laura Rogers	Joelle Johnson
New Policy for Vacation.docx	May 13	Laura Rogers	Brenda Cox
Paid Time Off.docx	May 13	Laura Rogers	Laura Rogers

And here's why this matters: SharePoint's new built-in [approvals](#) setting can send a simple approval with a message, but it's one-size-fits-all. With Power Automate input parameters, you completely control what data gets collected, what the email says, who it goes to, and what happens to the item afterward. More complex to setup, but more flexibility.

⚡ Take It Further: A One-Click Button with Quick Steps

For the grand finale, I connected this flow to the new **Quick Steps** feature in SharePoint:

1. Copy your flow's ID — it's the GUID in the flow's URL, between the last slash and the question mark.



2. In your library, create a new [Quick Step](#) that executes a flow **for selected files**, and paste in the flow ID.
3. You can even display the button **conditionally** — I set mine to only show when the Policy owner column is not empty, since the flow emails the policy owner.
4. Then add a **Quick Step column** to your view, pick your quick step, and give the button a color.

Now there's an easy, obvious button sitting right in the library view. One click, the input panel pops up, and the flow runs.

💰 One More Thing: Licensing

This question came up in the chat, so let's clear up a big misconception: you do *not* need a separate Power Automate license for any of this. Standard connectors like SharePoint and Outlook are included with something as simple as an E1 Microsoft 365 license. A separate Power Automate license only comes into play when you use **premium connectors**. I've been building flows for years on a standard license and have never been billed extra.

To view the underlying code (JSON) of a Power Automate flow, open your flow in the designer, click on the **menu (...)** for any specific trigger or action, and select **Peek code**. This displays the exact JSON structure behind your workflow. [\[1\]](#), [\[2\]](#)

How to Access the Full Flow Code

For complete, end-to-end flow definitions, you can view, edit, or migrate the entire JSON using these methods:

- **Export as Logic Apps:** Export your flow as a Logic Apps template to view the declarative JSON markup, which is ideal for deploying across different environments. [\[1\]](#)
- **Web-based Extensions:** Use community tools (such as Power Bookmarklets) to instantly grab the Get Flow JSON and format the code for viewing or AI consumption. [\[1\]](#)
- **Manage with Code:** View and work with cloud flows using code by utilizing the Power Platform CLI to manage workflow properties and run execution code. [\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#)

Typical JSON Code Structure

Power Automate actions operate as JSON objects. They are broken down into several distinct components: [\[1\]](#)

- **inputs:** The data or parameters required for the step, such as the target email address or the body of an API call.
- **runAfter:** Dictates the flow execution order by defining which previous step must finish and its status (e.g., Succeeded, Failed) before this action begins.
- **type:** Specifies the nature of the action (e.g., OpenApiConnection, Compose, Scope). [\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#), [\[5\]](#)

To work directly with your flow's configuration files or to integrate them into your developer projects, refer to the [Microsoft Learn Guide on Working with cloud flows using code](#).

Peek Code

What is the Peek Code Feature in Power Automate?

In Power Automate, "Peek code" (also known as code view) allows you to see the underlying JSON representation of a flow's actions and triggers. This feature is helpful for understanding the structure and configuration of your flows and their actions, as it displays the code that Power Automate is generating to automate your workflow.

How to Use it to Edit Flows in Power Automate?

Here is the step-by-step process that you need to follow to edit flows with the help of the Peek Code feature:

STEP 1: Open the flow you want to edit in Power Automate

STEP 2: Find the condition action you want to edit.

STEP 3: Click on the ellipsis (...) on the condition action

STEP 4: You will get the option to select Peek Code. Click on it.

You will get a small panel that shows the JSON for that condition.

STEP 5: The Peek Code panel will have fields like:

“left” and “right” – which compare the two values.

“operator” – the type of comparison.

STEP 6: Edit the values directly as required.

STEP 7: After you are done editing, simply close the peek code. Your changes will be saved automatically.

An Example of a Condition Within Peek Code

```
{
  "inputs": {
    "conditions": [
      {
        "left": "@triggerBody()?['Amount']",
        "operator": "greater",
        "right": 1000
      }
    ]
  }
}
```

Please make sure that you make the edits correctly, as a single typo can break the entire flow. For large complex flows, this would mean spending hours debugging. A smart idea will be to save the flow before you make the changes.

[Edit Peek Code Feature in Power Automate - Evolvous](#)

PL-200 Git Labs & Flows

Hey Co - Goal: create the simplest Cloud Flow with one variable to teach using parameters. PL-200 training seems to lag a bit on github Is there something in the Fabrikam zip that has a variable to show as working example and peek in code?

Where in the current labs does it cover this topic?

<https://github.com/MicrosoftLearning/PL-200-Power-Platform-Functional-Consultant> this tutorial seems like a good approach but once again seemed not to pull together as shown. <https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/how-to/trigger-flow>

Copilot

Hey Tim,

I looked through the PL-200 lab structure earlier, and I think you're bumping into a genuine gap.

The labs teach:

Dataverse

- Apps
- Flows
- Business Rules

but they don't do a great job of teaching:

Input Parameter

- Flow
- Output Parameter
- App

which is exactly the pattern used in the Canvas App → Flow tutorial you linked.microsoft

Where This Appears In The Labs

Closest matches:

Lab 4.1

Canvas Apps

Lab 6.1

Cloud Flows

The challenge is that they're taught separately. The student has to mentally connect:

Canvas App

with

Cloud Flow

to get to the Microsoft Learn scenario:

Canvas App

↓

Pass Parameter

↓

Power Automate

↓

Return Result

The GitHub labs definitely emphasize creating apps and flows, but not the "parameter passing" pattern as clearly as the Learn article.[microsoft+1](#)

Is There Something In Fabrikam?

Yes, probably.

When you imported:

FabrikamEnvironmental_*.zip

it imported not only:

Tables

Apps

Flows

but also:

Environment Variables

Power Automate Flows

Business Process Flows

inside the solution.github

The easiest investigation path is:

Solutions

↓

Fabrikam Environmental

↓

Environment Variables

Open:

Outcome Form Link

or

LOB Guid List

from your screenshot.

Then:

See where used

or search solution contents.

You can usually discover:

Flow
references variable

which gives a real-world example of parameterized configuration.

If You Want To "Peek In Code"

The easiest Flow to inspect is actually:

Parameter

↓

Compose

because the definition is tiny.

Open:

Flow

↓

Export

or

Peek code

(depending on designer availability)

You'll generally see JSON similar to:

JSON

```
{  
  "type": "string",  
  "name": "ProjectName"  
}
```

and later:

JSON

```
"inputs": "@triggerBody()['ProjectName']"
```

The student can immediately see:

Parameter Name

becoming

Dynamic Content Token

inside the Flow.

The Progression of What to Learn/When

Instead of the GitHub order:

Apps
Flows
Dataverse

I'd teach:

Level 1

Parameter
↓
Compose
Level 2

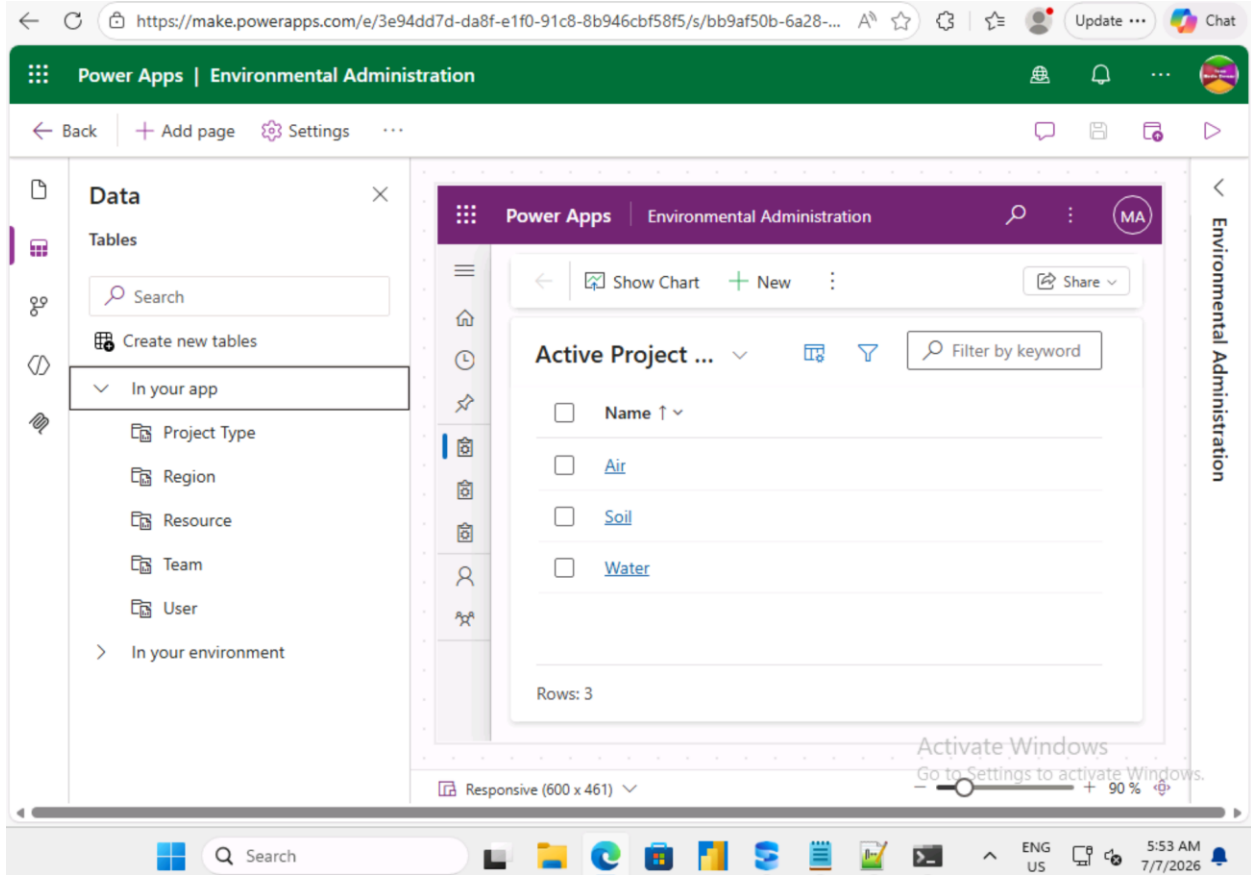
Parameter
↓
Dataverse
Level 3

Canvas App
↓
Flow Parameter
↓
Dataverse
Level 4

Canvas App
↓
Flow Parameter
↓
Approval
↓
Dataverse

That's the exact mental bridge between the Learn article and the real PL-200 world. Once students understand **"a flow is just a function that accepts parameters and performs actions"**, nearly everything in Power Automate starts making sense.

Screen Shots of Tools



Power Apps | Environmental Administration

Back Add page Settings

Data

Tables

Search

Create new tables

In your app

- Project Type
- Region
- Resource
- Team
- User

In your environment

Power Apps | Environmental Administration

Show Chart New

Share

Active Project ...

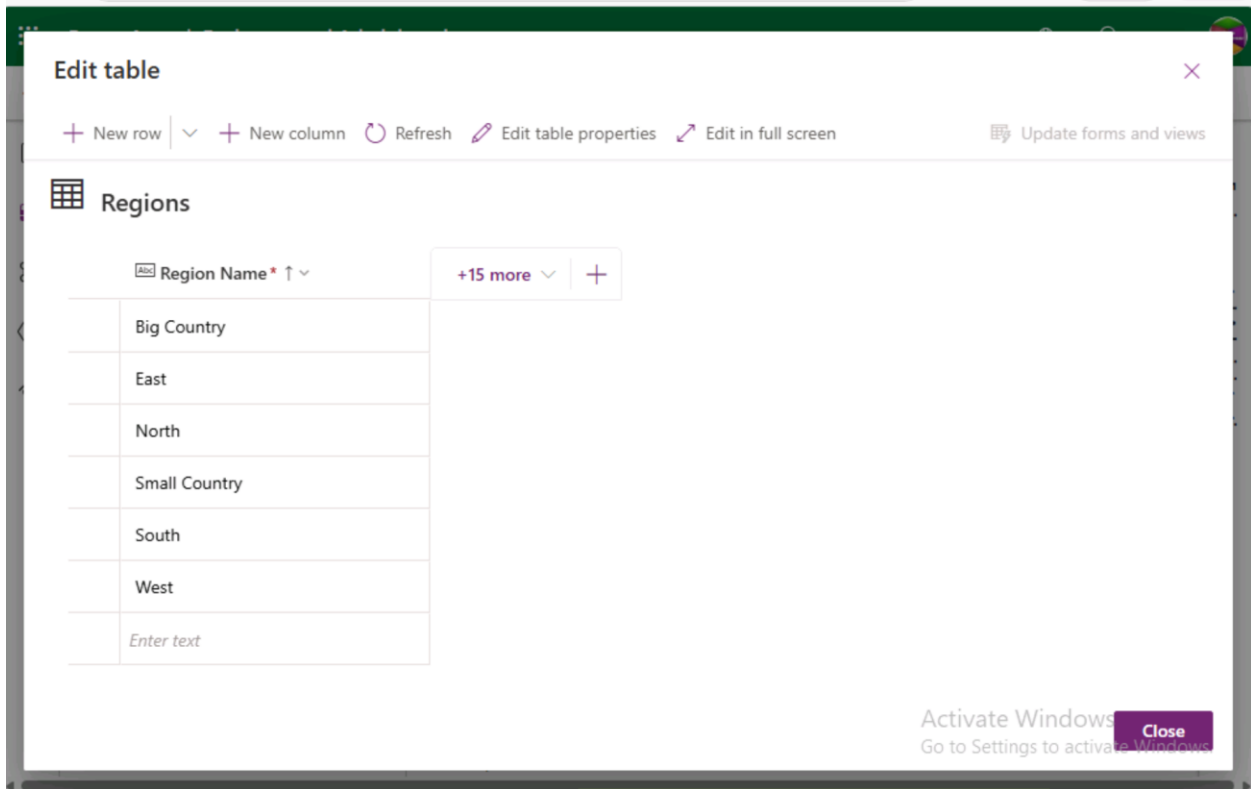
Filter by keyword

- ☐ Name ↑
- ☐ Air
- ☐ Soil
- ☐ Water

Rows: 3

Responsive (600 x 461)

Activate Windows
Go to Settings to activate Windows.



Edit table

New row New column Refresh Edit table properties Edit in full screen Update forms and views

Regions

Region Name * ↑

+15 more

Big Country
East
North
Small Country
South
West
Enter text

Activate Windows
Go to Settings to activate Windows.

Close

Reference

1. [Start a flow in a canvas app - Power Apps | Microsoft Learn](#)
2. [Define input and output parameters in Power Automate - Training | Microsoft Learn](#)
3. [Power Automate Input Parameters: A Form Inside Your Flows](#)
4. [Use expressions in conditions in Power Automate - Power Automate | Microsoft Learn](#)
5. [Work with cloud flows using code - Power Automate | Microsoft Learn](#)
6. [Edit Peek Code Feature in Power Automate - Evolvous](#)
7. [Explore the cloud flows designer - Power Automate | Microsoft Learn](#)